Unit4 - Orange Program Assignment

NAME: Sagar K SEC: G

SRN:PES2UG24CS424

Phase 1:

Screenshot 1A — test_objects

```
=== All integration tests completed ===
sagark@ubuntu:~/Desktop/OS_Orange$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
sagark@ubuntu:~/Desktop/OS_Orange$
```

Screenshot 1B — sharded object directory

```
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
sagark@ubuntu:~/Desktop/OS_Orange$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
sagark@ubuntu:~/Desktop/OS_Orange$
```

.

Phase 2:

Screenshot 2A — test_tree

```
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
sagark@ubuntu:~/Desktop/OS_Orange$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
sagark@ubuntu:~/Desktop/OS_Orange$
```

.

Screenshot 2B — raw binary of a tree object

```
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
sagark@ubuntu:~/Desktop/OS_Orange$ xxd .pes/objects/f5/902ee73b6aa25436f687067fc4cc0cb481cf416740d3c3c3ed6d9c4
de7b6c2 | head -20
00000000: 7472 6565 2039 3800 3130 3036 3434 2066  tree 98.100644 f
00000010: 696c 6531 2e74 7874 002c f8d8 3d9e e295  ile1.txt.,..=...
00000020: 43b3 4a87 7274 21fd ecb7 e3f3 a183 d337  C.J.rt!........7
00000030: 6390 25de 576d b9eb b431 3030 3634 3420  c.%.Wm...100644
00000040: 6669 6c65 322e 7478 7400 e00c 50e1 6a2d  file2.txt...P.j-
00000050: f38f 8d6b f809 e181 ad02 48da 6e67 19f3  ...k......H.ng..
00000060: 5f9f 7e65 d6f6 0619 9f7f                 _.~e......
sagark@ubuntu:~/Desktop/OS_Orange$
```

.

Phase 3:

Unit4 - Orange Program Assignment

NAME: Sagar K SEC: G

SRN:PES2UG24CS424

Screenshot 3A — init, add, status

```
sagark@ubuntu:~/Desktop/OS_Orange$ ./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt
./pes status
Initialized empty PES repository in .pes
Added 2 file(s)
Staged changes:
  new file:   file1.txt
  new file:   file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  test_sequence.sh
  untracked:  test_objects.c
  untracked:  test_tree
  untracked:  bye.txt
  untracked:  tree.c
  untracked:  test_tree.c
  untracked:  commit.c
  untracked:  commit.h
  untracked:  pes.c
  untracked:  Makefile
  untracked:  object.c
  untracked:  index.h
  untracked:  pes.h
  untracked:  hello.txt
  untracked:  index.c
  untracked:  tree.h
  untracked:  test_objects
  untracked:  README.md

sagark@ubuntu:~/Desktop/OS_Orange$
```

.

Screenshot 3B — index file

```
sagark@ubuntu:~/Desktop/OS_Orange$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776754066 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776754066 6 file2.txt
sagark@ubuntu:~/Desktop/OS_Orange$
```

.

Phase 4

Unit4 - Orange Program Assignment

NAME: Sagar K SEC: G

SRN:PES2UG24CS424

Screenshot 4A — log

```
sagark@ubuntu:~/Desktop/OS_Orange$ ./pes log
commit c4f40f0d03e9466c2dd456f85b65205a93380c13602662a1ddcb51dd522f63d4
Author: Sagar K PES2UG24CS424
Date:   1776753959

    Initial commit

sagark@ubuntu:~/Desktop/OS_Orange$
```

.

Screenshot 4B — object growth

```
sagark@ubuntu:~/Desktop/OS_Orange$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/c4/f40f0d03e9466c2dd456f85b65205a93380c13602662a1ddcb51dd522f63d4
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/f5/902ee73b6aa25436f687067fc4cc0cb481cf416740d3c3c3ed6d9c4de7b6c2
.pes/refs/heads/main
sagark@ubuntu:~/Desktop/OS_Orange$
```

.

Screenshot 4C — reference chain

```
.pes/refs/heads/main
sagark@ubuntu:~/Desktop/OS_Orange$ cat .pes/refs/heads/main
c4f40f0d03e9466c2dd456f85b65205a93380c13602662a1ddcb51dd522f63d4
sagark@ubuntu:~/Desktop/OS_Orange$ cat .pes/HEAD
ref: refs/heads/main
sagark@ubuntu:~/Desktop/OS_Orange$
```

.

Analysis Questions:

Q5.1 — Implementing `pes checkout <branch>`

Read the target branch file in `.pes/refs/heads/<branch>` to get the commit hash. Parse the commit to get its tree, then restore all files from that tree to the working directory. Update `.pes/HEAD` to point to the new branch and rewrite the index to match. The complexity lies in safely handling files that differ between branches without overwriting local changes.

Q5.2 — Detecting dirty working directory conflicts

Unit4 - Orange Program Assignment

NAME: Sagar K SEC: G
SRN:PES2UG24CS424

For each file in the index, recompute its SHA-256 hash from disk and compare it to the stored hash. If they differ, the file is locally modified. If that same file also differs between the current and target branch trees, checkout must be refused to prevent overwriting uncommitted changes.

Q5.3 — Detached HEAD

In detached HEAD, `.pes/HEAD` contains a raw commit hash instead of a branch reference. New commits are created normally but no branch pointer is updated, so switching branches makes those commits unreachable. To recover, create a new branch pointing to the last commit hash before switching: `git branch recovery-branch <hash>`.

Q6.1 — Garbage Collection Algorithm

Start from all branch refs, walk every commit chain, and mark all referenced blobs and trees as reachable using a hash set. Then scan all objects in `.pes/objects/` and delete any not in the reachable set. A hash set gives O(1) lookup, making this efficient even for large repositories.

Q6.2 — GC Race Condition

A commit writes a blob first, then the commit object. If GC runs between these two steps, it sees the blob as unreachable and deletes it, corrupting the repository. Git avoids this by never deleting objects newer than two weeks, ensuring in-progress commits are always protected.

Github Link:

https://github.com/sagark-05/PES2UG24CS424-pes-vcs.git